

Laboratory 6:

AN ENHANCED PROCESSOR WITH BRANCH INSTRUCTIONS

OBJECTIVES

- To extend the processor developed in Lab 5 by adding support for branch instructions.
- To learn how to interface basic input and output devices, including switches, LEDs, and 7-segment displays, with an FPGA chip, and to implement digital circuits using these peripherals.
- To develop and execute a simple application program based on the instruction set of the designed processor

PREPARATION FOR LAB 6

- Finish all the lab from 1-5.
- Students must do lab 6, then write the report of this project. The report should include all steps of your design: block diagram, waveform, RTL viewer, FSM, ...
- When reporting to the instructors, you will answer random questions to prove you understand the project.

REFERENCE

1. Intel FPGA training



Laboratory 6:

AN ENHANCED PROCESSOR WITH BRANCH INSTRUCTIONS

Requirements:

- In this lab, the *mvnz* instruction is no longer supported in the processor. In addition, two new instructions for program branching are required, as shown in Table 1.

Operation	Function performed
brne label	if $Q \neq 0$ then $PC \leftarrow \text{address of the instruction at label (Jump to label if } Q \neq 0)$
brlt label	if $Q < 0$ then $PC \leftarrow \text{address of the instruction at label (Jump to label if } Q < 0)$

Table 1: New instruction performed in the processor.

- Connect the enhanced processor from Lab 5 to two additional registers used for the 7-segment display and switches.

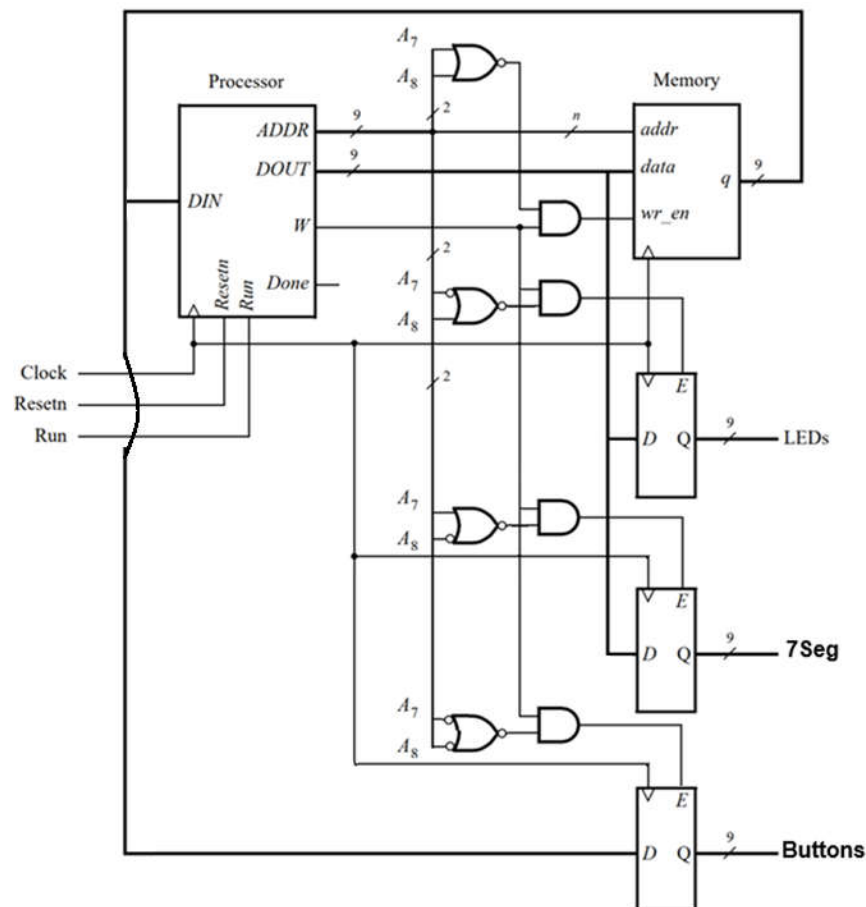


Figure 2: Connecting the enhanced processor to a memory and output register.



Laboratory 6:

AN ENHANCED PROCESSOR WITH BRANCH INSTRUCTIONS

- Write a program using 8 instructions `mv`, `mvi`, `add`, `sub`, `ld`, `st`, `brne` and `brlt` to read the button inputs and display the data on the 7-segment displays and LEDs. (More complex programs will receive higher grades)

Instruction:

A sample program used to multiple R0 with R1 using *brne* instruction:

Adress (PC)	Program	Machine code	Description
0x00	<code>mvi R0,#2</code>	001 000 000	$R0 \leftarrow 2$
0x01		000 000 010	
0x02	<code>mvi R1,#3</code>	001 001 000	$R1 \leftarrow 3$
0x03		000 000 011	
0x04	<code>mvi R2,#1</code>	001 010 000	$R2 \leftarrow 1$
0x05		000 000 001	
0x06	Loop <code>add R0,R0</code>	010 000 000	Start loop: $R0 \leftarrow R0 + R0$
0x07	<code>sub R1,R2</code>	011 001 010	$R1 \leftarrow R1 - 1$
0x08	<code>brne Loop</code>	110 000 000	If $G \neq 0$ then go to Loop
0x09		000 000 110	

In the program, the instruction **brne** Loop has the machine code **110 000 110**. The opcode for `brne` is 110, and the following 6 bits are ignore. The 9-bit machine code is used to store the address of the instruction labeled Loop.

The **brlt** instruction has a similar format, with the opcode 111. Using this format, the two branch instructions can jump to any where in memory (This type of jump is sometimes referred to as a absolute jump).

